Pulling It All Together

Let's look at an example of how Rails applies a number of Ruby features together to make the code you need to maintain more declarative. You'll see this example again in *Generating the Scaffold*. For now, we'll focus on the Ruby-language aspects of the example:

```
class CreateProducts < ActiveRecord::Migration[7.0]
  def change
    create_table :products do |t|
      t.string :title
      t.text :description
      t.string :image_url
»     t.decimal :price, precision: 8, scale: 2

      t.timestamps
    end
  end
end
```

Even if you didn't know any Ruby, you'd probably be able to decipher that this code creates a table named `products`. The fields defined when this table is created include `title`, `description`, `image_url`, and `price`, as well as a few timestamps (we'll describe these in Chapter 23, *Migrations*).

Now let's look at the same example from a Ruby perspective. We define a class named `CreateProducts`, which inherits from the versioned[35] `Migration` class from the `ActiveRecord` module, specifying that compatibility with Rails 7 is desired. We define one method, named `change`. This method calls the `create_table` method (defined in `ActiveRecord::Migration`), passing it the name of the table in the form of a symbol.

The call to `create_table` also passes a block that is to be evaluated before the table is created. This block, when called, is passed an object named `t`, which is used to accumulate a list of fields. Rails defines a number of methods on this object—methods named after common data types. These methods, when called, simply add a field definition to the ever-accumulating set of names.